Brainstorm Bouwcontrole — afhankelijkheden & alpha-ontwerp

Sep 28, 2026 · @Jeroen

Aanleiding

Deze brainstorm legt twee ideeën vast om met het team te bespreken en later in een

.

1. .
2. andere chat van dit project verder uit te werken.

- **E0.**
- **.**
- **. en nieuwe soort controle:** een artikel dat een ander artikel vereist.
- **Een alpha van het scherm:** vroeger dan het stappenplan voorziet, om nu al praktisch te testen.

Het document vervangt niets uit het stappenplan of de conflictanalyse. Het vult ze aan.

Nieuw idee: verplichte begeleidende artikels

Sommige artikels werken pas als een ander artikel mee in de order zit. Ontbreekt dat, dan moet Bouwcontrole het melden of tegenhouden.

Het geval dat het idee deed ontstaan

- NZXT-casefans in losse verpakking hebben geen controller bij zich.
- De controller zit wel in de triple pack van hetzelfde type, of wordt apart verkocht.
- Losse fans zonder aparte controller en zonder triple pack van hetzelfde type: de bouw is niet af te werken.

Waarom dit een ander soort controle is

	Bestaande regels	Deze regel
De vraag	Passen twee onderdelen uit de order bij elkaar?	Zit alles in de order wat een onderdeel nodig heeft?
Vuurt op	iets dat er is	iets dat ontbreekt

	Bestaande regels	Deze regel
Voorbeeld	`FAN-KST-MAAT`: fanmaat tegenover de behuizing	losse NZXT-fan zonder controller

Twee dingen in de conflictanalyse liggen er het dichtst bij:

- **De controle op beeldduitgang over de hele order.** Ook die zoekt iets dat ontbreekt: een grafische kern of een videokaart.
- **De eigenschap *controller meegeleverd* bij casefans.** Die schakelt nu alleen `FAN-MB-HEADERS` en `FAN-MB-RGB` uit. Hier zou ze voor het eerst zelf een melding veroorzaken.

Plaats in het ontwerp

De regel past in de bestaande opzet, waarin een conflict een verzameling voorwaarden is. Maar hij stelt vragen die de huidige regels nog niet stellen. Die vragen staan hier als richting, niet als uitwerking.

- **Wat is "hetzelfde type"?** Geen enkele bestaande regel kent een productfamilie. Is dat een eigenschap van het artikel, een koppeling tussen artikels, of nog iets anders?
- **Identificatie of eigenschap?** Het ontwerp zegt: nu voorwaarden op het Kraken-Id, later op eigenschappen. Bij welke van de twee hoort deze regel?
- **Welk onderdeeltype is een controller?** Een casefan, een eigen type, of een toebehoren zonder type?
- **Hoe zegt een voorwaarde "dit ontbreekt"?** Tot nu toe gaat elke voorwaarde over een onderdeel dat er is.
- **Welke ernst?** Oranje of rood. Dat raakt open vraag 4 uit het ontwerp: is rood werkelijk blokkerend?
- **Welke regel kleurt het scherm?** Het ontbrekende artikel heeft geen regel om te kleuren. Kleurt de fan, of komt er een aparte melding?

Richting: beschrijf eerst in gewone taal een paar orders die goed, fout en twijfelachtig zijn. Kijk dan of je bestaande voorwaarden ze uit elkaar kunnen houden. Waar dat niet lukt, zit het stuk dat ontbreekt.

Alpha-versie van het scherm

In het stappenplan komt het scherm pas in fase 14, na de tests en Identity. De alpha haalt er bewust een versie van naar voren. Dat werd beslist op 28 september 2026.

Waarom nu

- Praktisch testen op de werkvloer, ook met de echte scanner.
- Met het team zien wat ontbreekt, voordat de database en de regels vastliggen.
- Iets hebben om te tonen.

Wat de alpha is

- Een los prototype van het ene scherm uit de mockup.
- Werkend met verzonnen gegevens.
- Een plek om nieuwe ideeën zoals de verplichte begeleidende artikels uit te proberen.

Wat de alpha niet is

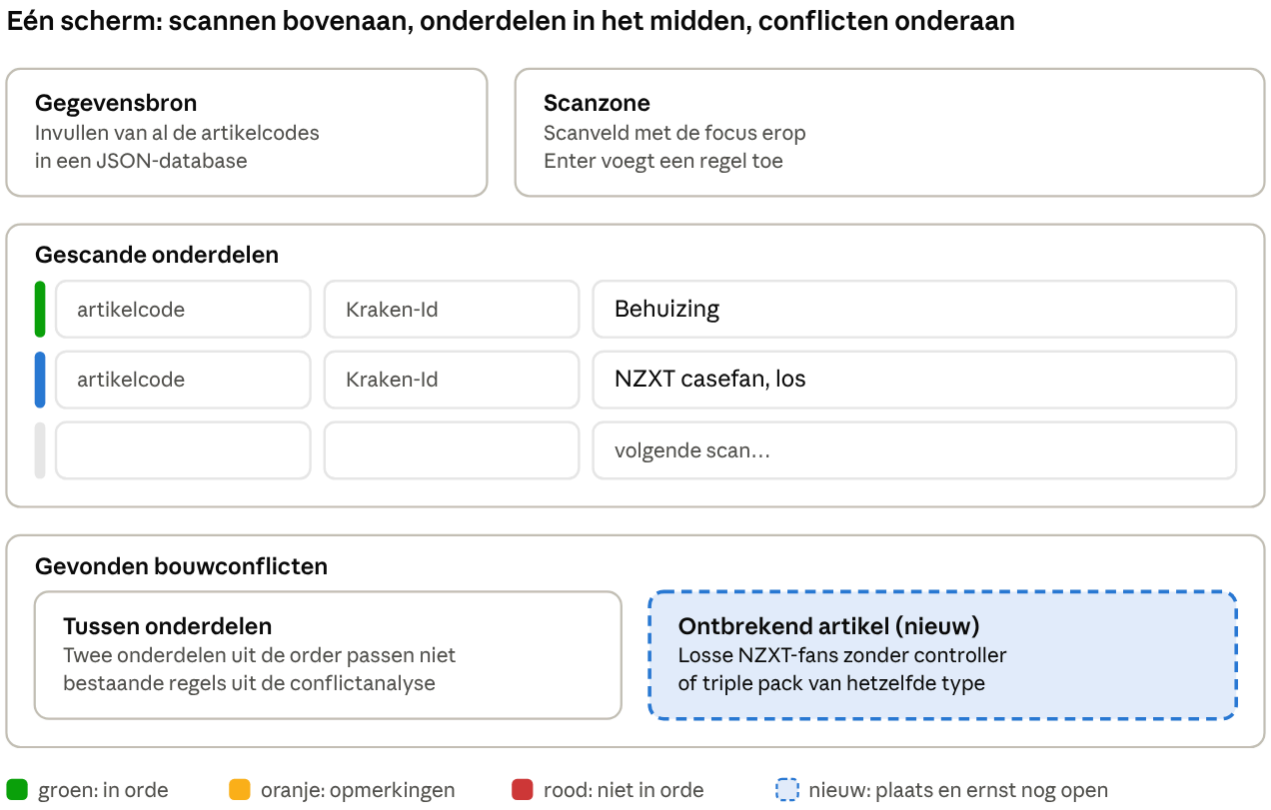
- Geen begin van fase 14. De echte views komen later in MVC, met ViewModels.
- Geen conflict-engine. De controles worden nagebootst; de echte engine is fase 11.
- Geen koppeling met de API of de database.

Welke afspraak wordt geplooid

De volgorde van de leerstof. Deel 1 blijft verder hetzelfde: aan het einde van fase 11 een werkende demo via Scalar. De harde regels blijven voor het project gelden. Enkel voor de start van de alpha, in de laatste twee secties, zijn ze op vraag opgeheven.

Wat de alpha moet tonen

De alpha toont het ene scherm uit de mockup, plus een plek voor de nieuwe melding over een ontbrekend artikel.



alpha-scherm · vier zones uit de mockup, één nieuw vak

Het gestippelde vak is het enige nieuwe stuk. Waar het precies komt, en in welke kleur, beslist het team.

Dit ontwerp geldt voor zowel de HTML- als de Razor-variant.

Toestanden die het scherm moet kunnen tonen

Toestand	Wat de gebruiker ziet
In orde	groene regel
Opmerking	oranje regel, met uitleg in het conflictpaneel
Niet in orde	rode regel, met uitleg in het conflictpaneel
Onbekende code	een melding dat de scan niets oplevert
Ontbrekend artikel	nieuw, vorm nog open

Scenario's om met het team door te lopen

1. Losse NZXT-fans met een aparte controller.
2. Losse NZXT-fans met een triple pack van hetzelfde type.
3. Alleen losse NZXT-fans.

4. Losse NZXT-fans met een triple pack van een ander type.
5. Een code die niet bestaat.

Bij 1 en 2 verwacht je geen melding, bij 3 wel. Wat 4 moet geven, weet het team beter dan de software.

Open vragen voor het team

Deze vragen moeten uit de praktijk komen, niet uit de software. Vink af wat beantwoord is en noteer het antwoord erbij.

Over verplichte begeleidende artikels

- [ ] Welke andere artikels hebben een verplichte partner die niet in de verpakking zit?
- [ ] Hoe herkent het team vandaag "hetzelfde type": op naam, op reeks, op iets anders?
- [ ] Stuurt een controller van een ander type of een andere reeks de losse fans ook aan?
- [ ] Hoeveel fans kan één controller aan, en moet Bouwcontrole dat tellen?
- [ ] Melding of blokkade: mag iemand doorgaan zonder het ontbrekende artikel?

Over de alpha

- [ ] Blijft de artikelcode op het scherm naast het Kraken-Id? Dat is open vraag 1 uit het ontwerp.
- [ ] Komt een order binnen door stuk per stuk te scannen, of uit een bestand of ordernummer? Dat is open vraag 2.
- [ ] HTML of Razor als vorm voor de alpha? Beide varianten staan in de laatste twee secties.

Start van het paginaontwerp

Begin met één statische HTML-pagina met verzonnen gegevens, los van de .NET-solution. Zo test je het scherm met de echte scanner zonder één laag van het stappenplan aan te raken.

> Op vraag gelden de harde regels in deze sectie niet: hier staat wel code. Het is een startpunt voor de alpha, geen deel van het stappenplan.

Waarom HTML en niet meteen Razor

- Een Razor-view is HTML en CSS met C# ertussen. De opmaak die je nu maakt, neem je in fase 14 grotendeels mee.

- Er is geen controller, ViewModel of database nodig, dus de volgorde van de C#-lessen blijft intact.
- Het sluit aan bij je webcursus.
- Het team opent het met een dubbelklik in de browser.

Stappen

1. Maak een map `alpha/` in de repo, naast de solution en niet erin.
2. Zet er vier bestanden in: `index.html`, `alpha.css`, `onderdelen.js` en `alpha.js`.
3. Bouw de zones uit de mockup met CSS Grid, volgens het skelet hieronder.
4. Vul `onderdelen.js` met een handvol verzonnen artikels, waaronder de NZXT-gevallen.
5. Laat het scanveld reageren op Enter: opzoeken, een regel toevoegen, controleren.
6. Test met de echte scanner. De meeste USB-scanners typen de code als een toetsenbord en sluiten af met Enter, dan werkt stap 5 zonder extra werk.
7. Loop de vijf scenario's door met het team en noteer wat ontbreekt.

index.html: het skelet

```
<!DOCTYPE html>
<html lang="nl">
<head>
  <meta charset="utf-8">
  <title>Bouwcontrole – alpha</title>
  <link rel="stylesheet" href="alpha.css">
</head>
<body>
  <main class="scherm">
    <section class="bron">
      <h2>Gegevensbron</h2>
    </section>
    <section class="scanzone">
      <label for="scan">Scanzone</label>
      <input id="scan" autofocus autocomplete="off">
      <p id="scanmelding" class="scanmelding"></p>
    </section>
    <section class="onderdelen">
      <h2>Gescande onderdelen</h2>
      <ul id="lijst"></ul>
    </section>
    <section class="conflicten">
      <h2>Gevonden bouwconflicten</h2>
      <div class="vakken">
        <div class="vak">
          <h3>Tussen onderdelen</h3>
```

```
          <ul id="tussen"></ul>
        </div>
        <div class="vak nieuw">
          <h3>Ontbrekend artikel</h3>
          <ul id="ontbrekend"></ul>
        </div>
      </div>
    </section>
  </main>
  <script src="onderdelen.js"></script>
  <script src="alpha.js"></script>
</body>
</html>
```

alpha.css: de indeling

```
.scherm {
  display: grid;
  grid-template-columns: 1fr 2fr;
  grid-template-areas:
    "bron        scanzone"
    "onderdelen  onderdelen"
    "conflicten  conflicten";
  gap: 1rem;
  padding: 1rem;
}
.bron       { grid-area: bron; }
.scanzone   { grid-area: scanzone; }
.onderdelen { grid-area: onderdelen; }
.conflicten { grid-area: conflicten; }

/* klassen naar de ernst, niet naar de kleur */
.ok        { border-left: 6px solid green; }
.opmerking { border-left: 6px solid orange; }
.nok       { border-left: 6px solid red; }

/* drie velden per regel, zoals in de mockup */
.onderdelen ul { list-style: none; padding: 0; }

.regel {
  display: grid;
  grid-template-columns: 1fr 1fr 2fr;
  gap: 0.5rem;
  margin-bottom: 0.5rem;
  padding-left: 0.5rem;
}
```

```
.veld {
  border: 1px solid #ccc;
  border-radius: 6px;
  padding: 0.4rem 0.6rem;
}

/* twee vakken in het conflictpaneel */
.vakken {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 1rem;
}
.vak {
  border: 1px solid #ccc;
  border-radius: 8px;
  padding: 0.75rem;
}
.vak.nieuw { border-style: dashed; }
```

De klassen heten naar de ernst, niet naar de kleur. Zo blijft de afspraak uit het ontwerp overeind: de kleur is een beslissing van het scherm.

onderdelen.js: verzonnen gegevens

```
// Verzonnen artikels, enkel voor de alpha
const onderdelen = [
  { krakenId: "K-1001", artikelcode: "ART-1001", naam: "NZXT casefan, los",
type: "casefan",    reeks: "A", controllerMeegeleverd: false },
  { krakenId: "K-1002", artikelcode: "ART-1002", naam: "NZXT casefan, triple
pack",   type: "casefan",    reeks: "A", controllerMeegeleverd: true },
  { krakenId: "K-1003", artikelcode: "ART-1003", naam: "NZXT controller",
type: "controller", reeks: "A" },
  { krakenId: "K-1004", artikelcode: "ART-1004", naam: "NZXT casefan, triple
pack B", type: "casefan",    reeks: "B", controllerMeegeleverd: true },
  { krakenId: "K-2001", artikelcode: "ART-2001", naam: "Behuizing",
type: "behuizing" }
];
```

De gegevens staan in een script en niet in een `.json`-bestand. Een pagina die je met een dubbelklik opent, mag in de meeste browsers geen los bestand inlezen met `fetch`. Het veld `reeks` is een noodoplossing voor de alpha, geen antwoord op de vraag naar "hetzelfde type".

alpha.js: het scanveld

```
const scan = document.getElementById("scan");
const lijst = document.getElementById("lijst");
const order = [];

scan.addEventListener("keydown", (e) => {
  if (e.key !== "Enter") return;
  const code = scan.value.trim();
  scan.value = "";

  const onderdeel = onderdelen.find(o => o.krakenId === code);
  if (!onderdeel) {
    // toon: onbekende code
    return;
  }

  order.push(onderdeel);
  toonRegel(onderdeel);
  controleer(order);
});
```

Jouw volgende stuk

- `toonRegel` maakt een regel in de lijst met drie velden: artikelcode, Kraken-Id en naam. De CSS verwacht een `li` met de klassen `regel` en de ernst, en een `span` met klasse `veld` per veld.
- `controleer` loopt de order af en vult `#tussen` en `#ontbrekend` apart. Begin met scenario 3: alleen losse fans.
- Een onbekende code komt in `#scanmelding`, niet in het conflictpaneel.

Variant: start in Razor

Dezelfde alpha kan ook in Razor, in een apart wegwerpproject naast je cursussolution. Het scannen gebeurt dan zonder JavaScript: elke scan is een formulier dat naar de controller gaat.

> Ook hier gelden de harde regels op vraag niet. Dit is een startpunt voor de alpha, geen deel van het stappenplan.

Wat anders is dan de HTML-versie

- De nagebootste controle zit in C#, in een Service, waar later ook de echte engine komt.
- Geen JavaScript: na elke scan laadt de pagina opnieuw.
- Het project moet draaien om het te tonen.

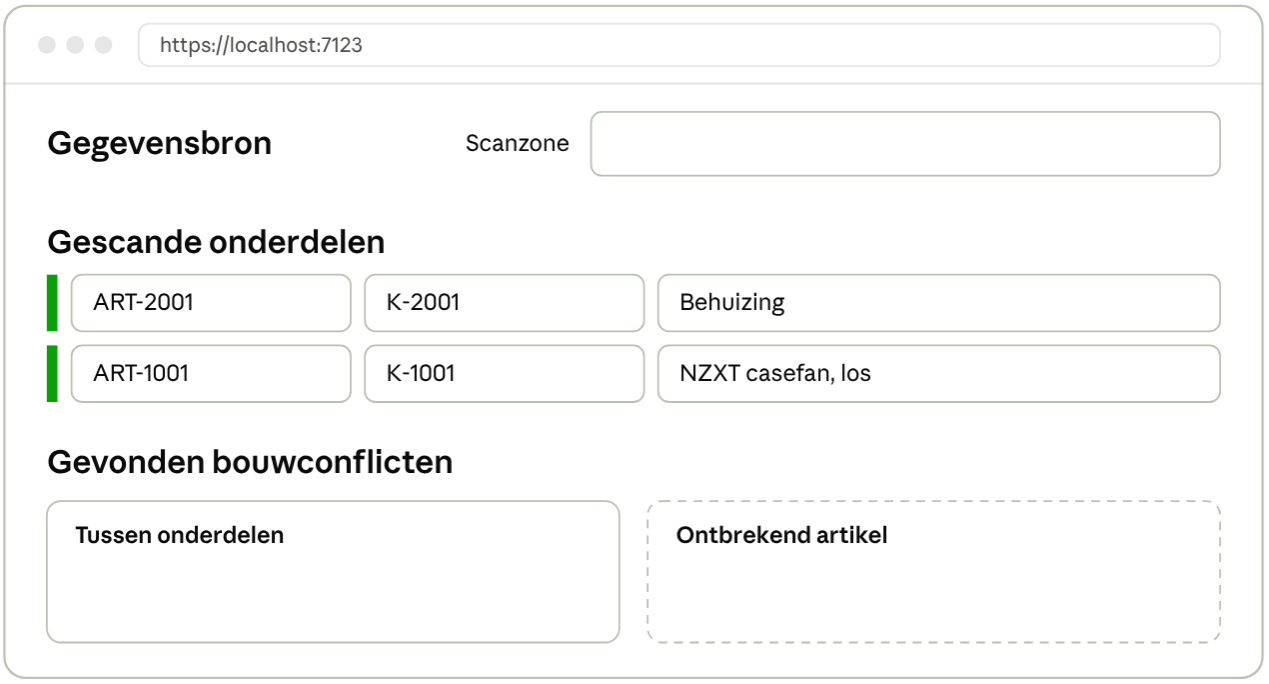
- Je leest .NET Development les 09 en 10 nu al, in plaats van in fase 14.

Stappen

1. Maak het project in Visual Studio met het sjabloon *ASP.NET Core Web App (Model-View-Controller)*, of met `dotnet new mvc -n Alternate.Bouwcontrole.Alpha`. Kies .NET 10 en geef het een eigen solution naast je cursussolution.
2. Maak de mappen `Services` en `ViewModels`. `Models`, `Controllers` en `Views` staan er al.
3. Voeg de bestanden hieronder toe.
4. Voeg `@using Alternate.Bouwcontrole.Alpha.ViewModels` toe aan `Views/_ViewImports.cshtml`.
5. Haal in `Views/Shared/_Layout.cshtml` het `header`-blok met de navigatiebalk en het `footer`-blok weg. Dan staat er niets rond je scherm dat niet in de mockup zit.
6. Kopieer de CSS uit de HTML-versie naar `wwwroot/css/site.css`.
7. Start met F5 of `dotnet run`, test met de scanner en loop de vijf scenario's door.

Zo ziet het eruit na het starten

Na het starten: twee scans, de controle nog leeg



alpha in Razor · na twee scans, controle nog leeg

Waar het echte scherm afwijkt van deze afbeelding:

- Alle regels zijn groen en de twee vakken blijven leeg, zolang de controles niet geschreven zijn.

- Na elke scan laadt de pagina even opnieuw, omdat het formulier naar de controller gaat.
- De lettertypes en de titelgroottes komen van Bootstrap uit het sjabloon. Het scanveld heeft de gewone opmaak van de browser.

Models: het onderdeel en de ernst

```
// Models/Onderdeel.cs
namespace Alternate.Bouwcontrole.Alpha.Models;

public class Onderdeel
{
    public string KrakenId { get; set; } = "";
    public string Artikelcode { get; set; } = "";
    public string Naam { get; set; } = "";
    public string Type { get; set; } = "";
    public string? Reeks { get; set; }
    public bool ControllerMeegeleverd { get; set; }
}

// Models/Ernst.cs
namespace Alternate.Bouwcontrole.Alpha.Models;

public enum Ernst { InOrde, Opmerking, NietInOrde }
```

Services: verzonnen gegevens

```
// Services/IOnderdelenService.cs
using Alternate.Bouwcontrole.Alpha.Models;

namespace Alternate.Bouwcontrole.Alpha.Services;

public interface IOnderdelenService
{
    Onderdeel? Zoek(string krakenId);
}

// Services/OnderdelenService.cs
using Alternate.Bouwcontrole.Alpha.Models;

namespace Alternate.Bouwcontrole.Alpha.Services;

public class OnderdelenService : IOnderdelenService
{
    // Verzonnen artikels, enkel voor de alpha
    private readonly List<Onderdeel> _onderdelen = new()
```

```
    {
        new() { KrakenId = "K-1001", Artikelcode = "ART-1001", Naam = "NZXT
casefan, los",           Type = "casefan",    Reeks = "A" },
        new() { KrakenId = "K-1002", Artikelcode = "ART-1002", Naam = "NZXT
casefan, triple pack",   Type = "casefan",    Reeks = "A",
ControllerMeegeleverd = true },
        new() { KrakenId = "K-1003", Artikelcode = "ART-1003", Naam = "NZXT
controller",             Type = "controller", Reeks = "A" },
        new() { KrakenId = "K-1004", Artikelcode = "ART-1004", Naam = "NZXT
casefan, triple pack B", Type = "casefan",    Reeks = "B",
ControllerMeegeleverd = true },
        new() { KrakenId = "K-2001", Artikelcode = "ART-2001", Naam =
"Behuizing",                  Type = "behuizing" }
    };

    public Onderdeel? Zoek(string krakenId) =>
        _onderdelen.FirstOrDefault(o => o.KrakenId == krakenId);
}
```

Services: de nagebootste controle

```
// Services/IControleService.cs
using Alternate.Bouwcontrole.Alpha.Models;

namespace Alternate.Bouwcontrole.Alpha.Services;

public interface IControleService
{
    List<string> ControleerTussenOnderdelen(List<Onderdeel> order);
    List<string> ControleerOntbrekend(List<Onderdeel> order);
}

// Services/ControleService.cs
using Alternate.Bouwcontrole.Alpha.Models;

namespace Alternate.Bouwcontrole.Alpha.Services;

public class ControleService : IControleService
{
    public List<string> ControleerTussenOnderdelen(List<Onderdeel> order)
    {
        var meldingen = new List<string>();
        // Jouw stuk: twee onderdelen die niet bij elkaar passen
        return meldingen;
    }
```

```
    public List<string> ControleerOntbrekend(List<Onderdeel> order)
    {
        var meldingen = new List<string>();
        // Jouw stuk: het NZXT-geval, te beginnen met scenario 3
        return meldingen;
    }
}
```

De twee methodes volgen de twee vakken van de mockup. Ze zijn een snelle oplossing voor de alpha, geen antwoord op de ontwerpvraag hoe een voorwaarde "dit ontbreekt" uitdrukt.

ViewModels: wat het scherm nodig heeft

```
// ViewModels/BouwcontroleViewModel.cs
namespace Alternate.Bouwcontrole.Alpha.ViewModels;

public class BouwcontroleViewModel
{
    public string? Scan { get; set; }
    public string? ScanMelding { get; set; }
    public List<string> Gescand { get; set; } = new();
    public List<RegelViewModel> Regels { get; set; } = new();
    public List<string> ConflictenTussenOnderdelen { get; set; } = new();
    public List<string> OntbrekendeArtikels { get; set; } = new();
}

// ViewModels/RegelViewModel.cs
using Alternate.Bouwcontrole.Alpha.Models;

namespace Alternate.Bouwcontrole.Alpha.ViewModels;

public class RegelViewModel
{
    public string Artikelcode { get; set; } = "";
    public string KrakenId { get; set; } = "";
    public string Naam { get; set; } = "";
    public Ernst Ernst { get; set; }
}
```

Controllers: elke scan komt hier binnen

```
// Controllers/BouwcontroleController.cs
using Alternate.Bouwcontrole.Alpha.Models;
using Alternate.Bouwcontrole.Alpha.Services;
using Alternate.Bouwcontrole.Alpha.ViewModels;
```

```
using Microsoft.AspNetCore.Mvc;

namespace Alternate.Bouwcontrole.Alpha.Controllers;

public class BouwcontroleController : Controller
{
    private readonly IOnderdelenService _onderdelenService;
    private readonly IControleService _controleService;

    public BouwcontroleController(IOnderdelenService onderdelenService,
IControleService controleService)
    {
        _onderdelenService = onderdelenService;
        _controleService = controleService;
    }

    [HttpGet]
    public IActionResult Index()
    {
        return View(new BouwcontroleViewModel());
    }

    [HttpPost]
    public IActionResult Index(BouwcontroleViewModel invoer)
    {
        var model = new BouwcontroleViewModel { Gescand = invoer.Gescand };
        var scan = invoer.Scan?.Trim();

        if (!string.IsNullOrEmpty(scan))
        {
            if (_onderdelenService.Zoek(scan) != null)
                model.Gescand.Add(scan);
            else
                model.ScanMelding = $"Onbekende code: {scan}";
        }

        var order = new List<Onderdeel>();
        foreach (var krakenId in model.Gescand)
        {
            var onderdeel = _onderdelenService.Zoek(krakenId);
            if (onderdeel == null) continue;

            order.Add(onderdeel);
            model.Regels.Add(new RegelViewModel
            {
                Artikelcode = onderdeel.Artikelcode,
```

```
                KrakenId = onderdeel.KrakenId,
                Naam = onderdeel.Naam,
                Ernst = Ernst.InOrde // voorlopig, tot de controle dit
beslist
            });
        }

        model.ConflictenTussenOnderdelen =
_controleService.ControleerTussenOnderdelen(order);
        model.OntbrekendeArtikels =
_controleService.ControleerOntbrekend(order);

        // Anders zet de tag helper de vorige scan terug in het scanveld
        ModelState.Clear();
        return View(model);
    }
}
```

Program.cs: services registreren en de startpagina

```
// Bovenaan
using Alternate.Bouwcontrole.Alpha.Services;

// Vóór builder.Build()
builder.Services.AddScoped<IOnderdelenService, OnderdelenService>();
builder.Services.AddScoped<IControleService, ControleService>();

// In MapControllerRoute: Home wordt Bouwcontrole
pattern: "{controller=Bouwcontrole}/{action=Index}/{id?}"
```

Views: het scherm

```
@* Views/Bouwcontrole/Index.cshtml *@
@model BouwcontroleViewModel

<div class="scherm">
    <section class="bron">
        <h2>Gegevensbron</h2>
    </section>

    <section class="scanzone">
        <form asp-action="Index" method="post">
            @foreach (var krakenId in Model.Gescand)
            {
                <input type="hidden" name="Gescand" value="@krakenId" />
            }
```

```
            <label asp-for="Scan">Scanzone</label>
            <input asp-for="Scan" autofocus autocomplete="off" />
        </form>
        @if (Model.ScanMelding != null)
        {
            <p class="scanmelding">@Model.ScanMelding</p>
        }
    </section>

    <section class="onderdelen">
        <h2>Gescande onderdelen</h2>
        <ul>
            @foreach (var regel in Model.Regels)
            {
                <li class="regel @CssKlasse(regel.Ernst)">
                    <span class="veld">@regel.Artikelcode</span>
                    <span class="veld">@regel.KrakenId</span>
                    <span class="veld">@regel.Naam</span>
                </li>
            }
        </ul>
    </section>

    <section class="conflicten">
        <h2>Gevonden bouwconflicten</h2>
        <div class="vakken">
            <div class="vak">
                <h3>Tussen onderdelen</h3>
                <ul>
                    @foreach (var melding in
Model.ConflictenTussenOnderdelen)
                    {
                        <li>@melding</li>
                    }
                </ul>
            </div>
            <div class="vak nieuw">
                <h3>Ontbrekend artikel</h3>
                <ul>
                    @foreach (var melding in Model.OntbrekendeArtikels)
                    {
                        <li>@melding</li>
                    }
                </ul>
            </div>
        </div>
```

```
    </section>
</div>

@functions {
    // De kleur is een beslissing van het scherm: ernst in, CSS-klasse uit
    string CssKlasse(Ernst ernst) => ernst switch
    {
        Ernst.Opmerking => "opmerking",
        Ernst.NietInOrde => "nok",
        _ => "ok"
    };
}
```

De buitenste tag is een `div` en geen `main`. De layout van het sjabloon heeft er zelf al een, en twee in elkaar is geen geldige HTML.

Hoe het scannen werkt

- Het scanveld is het enige tekstveld in een formulier zonder knop. Een Enter verstuurt dan het formulier, en de scanner geeft die Enter vanzelf.
- De Kraken-Id's die al gescand zijn, gaan mee als verborgen velden. De controller bouwt bij elke scan de hele order opnieuw op.
- `ModelState.Clear()` is nodig: zonder die regel zet de tag helper de vorige scan terug in het veld.
- Een onbekende code is geen bouwconflict. Die melding komt daarom bij het scanveld, niet in het conflictpaneel.

Jouw volgende stuk

- `ControleerOntbrekend` in de `ControleService`: boots het NZXT-geval na, te beginnen met scenario 3.
- `ControleerTussenOnderdelen` blijft voorlopig leeg. Daar komen later de bestaande regels.
- Nu zet de controller elke regel op `InOrde`. Wat moeten de controles teruggeven zodat ook de juiste regel kleurt? Dat is dezelfde vraag als bij de conflict-engine in het ontwerp.

Leeg sjabloon voor de Razor-alpha

Dezelfde bestanden als de uitgewerkte versie hierboven, maar leeg. Per bestand staan er alleen de namespace, de klasse of interface, en één regel die zegt waarvoor het bestand dient. De inhoud schrijf je zelf. De bestanden zitten ook in de zip *Bouwcontrole Alpha sjabloon V1*.

> Op vraag: alleen lege klassen, zonder eigenschappen en zonder methodes. De uitgewerkte versie hierboven blijft staan.

De mappen

```
Alternate.Bouwcontrole.Alpha
├── GlobalUsings.cs
├── Controllers/
│   └── BouwcontroleController.cs
├── Models/
│   ├── Ernst.cs
│   └── Onderdeel.cs
├── Services/
│   ├── IControleService.cs
│   ├── ControleService.cs
│   ├── IOnderdelenService.cs
│   └── OnderdelenService.cs
├── ViewModels/
│   ├── BouwcontroleViewModel.cs
│   └── RegelViewModel.cs
└── Views/
    └── Bouwcontrole/
        └── Index.cshtml
```

GlobalUsings.cs

```
// Usings die in elk .cs-bestand van het project gelden.
// Views halen de hunne uit Views/_ViewImports.cshtml, niet uit dit bestand.
global using Alternate.Bouwcontrole.Alpha.Models;
global using Alternate.Bouwcontrole.Alpha.Services;
global using Alternate.Bouwcontrole.Alpha.ViewModels;
global using Microsoft.AspNetCore.Mvc;
```

De lege klassen

```
// Models/Onderdeel.cs
namespace Alternate.Bouwcontrole.Alpha.Models;

// Eén artikel uit de onderdelenlijst, zoals het gescand en opgezocht wordt.
public class Onderdeel
{
}

// Models/Ernst.cs
namespace Alternate.Bouwcontrole.Alpha.Models;
```

```
// Hoe zwaar een bevinding weegt. Het scherm maakt er een kleur van.
public enum Ernst
{
}

// Services/IOnderdelenService.cs
namespace Alternate.Bouwcontrole.Alpha.Services;

// Het contract om een onderdeel op te zoeken aan de hand van wat er gescand
werd.
public interface IOnderdelenService
{
}

// Services/OnderdelenService.cs
namespace Alternate.Bouwcontrole.Alpha.Services;

// Houdt de verzonnen onderdelen van de alpha bij en zoekt erin.
public class OnderdelenService : IOnderdelenService
{
}

// Services/IControleService.cs
namespace Alternate.Bouwcontrole.Alpha.Services;

// Het contract voor de nagebootste controles: tussen onderdelen en
ontbrekende artikels.
public interface IControleService
{
}

// Services/ControleService.cs
namespace Alternate.Bouwcontrole.Alpha.Services;

// Bootst de controles na. De echte conflict-engine komt pas in fase 11.
public class ControleService : IControleService
{
}

// ViewModels/BouwcontroleViewModel.cs
namespace Alternate.Bouwcontrole.Alpha.ViewModels;

// Alles wat het ene scherm nodig heeft: de scan, de gescande regels en de
meldingen.
public class BouwcontroleViewModel
```

```
{
}

// ViewModels/RegelViewModel.cs
namespace Alternate.Bouwcontrole.Alpha.ViewModels;

// Eén regel in de lijst met gescande onderdelen.
public class RegelViewModel
{
}

// Controllers/BouwcontroleController.cs
namespace Alternate.Bouwcontrole.Alpha.Controllers;

// Ontvangt elke scan, vraagt de services om hulp en geeft het ViewModel door
aan de view.
public class BouwcontroleController : Controller
{
}
```

De view

```
@* Het ene scherm uit de mockup: scanzone, gescande onderdelen en gevonden
bouwconflicten. *@
@model BouwcontroleViewModel
```

Wat nog buiten deze bestanden moet gebeuren

- `Views/_ViewImports.cshtml`: een `@using` voor de ViewModels. Doe dit vóór je de eerste keer bouwt, anders kent de view het ViewModel niet en krijg je een compileerfout.
- `Program.cs`: de twee services registreren, en de standaardroute laten starten bij Bouwcontrole in plaats van bij Home.
- `Views/Shared/_Layout.cshtml`: het `header`-blok met de navigatiebalk en het `footer`-blok weghalen.
- `wwwroot/css/site.css`: de CSS uit de HTML-versie.

Hoe de C# en de view samenhangen

1. Het adres `/Bouwcontrole` komt via de route bij `BouwcontroleController` en de actie `Index`.
2. De controller vult een `BouwcontroleViewModel` en geeft het mee aan de view.
3. MVC zoekt de view op naam: de map heet zoals de controller zonder "Controller", het bestand zoals de actie.

4. De view zegt met `@model` welk ViewModel hij verwacht, en toont het.
5. Het formulier stuurt de scan terug naar dezelfde actie. Model binding vult dan het ViewModel opnieuw.

Een apart C#-bestand achter elke pagina bestaat alleen bij Razor Pages. Hier is de controller dat achterliggende stuk.

Goed om te weten

- `GlobalUsings.cs` werkt alleen voor `.cs`-bestanden. Views halen hun usings uit `_ViewImports.cshtml`.
- Met `GlobalUsings.cs` mag je de `using`-regels bovenaan de uitgewerkte versie weglaten.
- Laat `HomeController` en `Models/ErrorViewModel.cs` uit het sjabloon staan. De foutpagina van het sjabloon gebruikt ze.